Question 1

Find pairs of products that appear in the same order and show how many different orders they share. The result should include the first product's name, the second product's name and the number of shared orders. The most frequent pair should appear first.

A product must not be paired with itself, and only one of A–B and B–A should be shown.

```
SELECT
    pp1.product_name AS [First Product],
    pp2.product_name AS [Second Product],
    COUNT(DISTINCT soi1.order_id) AS [Orders Together]
FROM sales.order_items AS soi1
INNER JOIN sales.order_items AS soi2
    ON soi1.order_id = soi2.order_id
    AND soi1.product_id < soi2.product_id
INNER JOIN production.products AS pp1
    ON soi1.product_id = pp1.product_id
INNER JOIN production.products AS pp2
    ON soi2.product_id = pp2.product_id
GROUP BY
    pp1.product_id, pp1.product_name,
    pp2.product_id, pp2.product_name
ORDER BY [Orders Together] DESC;
```

Question 2

If a product has 0 units in stock in one store and at least 10 in another, match these stores. The result should include the product name, the name of the store that can send stock, its stock quantity and the name of the store that will receive it. Sort by the sending store's quantity in descending order.

```
SELECT
    pp.product_name AS [Product Name],
    ss1.store_name AS [Sending Store],
    ps1.quantity AS [Sending Store Quantity],
    ss2.store_name AS [Receiving Store]
FROM production.stocks AS ps1
INNER JOIN production.stocks AS ps2
    ON ps1.product_id = ps2.product_id
    AND ps1.store_id != ps2.store_id
INNER JOIN production.products AS pp
    ON ps1.product_id = pp.product_id
INNER JOIN sales.stores AS ss1
    ON ps1.store_id = ss1.store_id
INNER JOIN sales.stores AS ss2
    ON ps2.store_id = ss2.store_id
WHERE ps1.quantity >= 10
    AND ps2.quantity = 0
ORDER BY ps1.quantity DESC;
```

Question 3

List the customers who have placed orders from at least two different stores. The result should include the customer ID, first name, last name, number of different stores ordered from and total number of orders. Sort by the number of different stores in descending order; in case of a tie, the customer with more total orders should come first.

```
SELECT
    sc.customer_id,
    sc.first_name,
    sc.last_name,
    COUNT(DISTINCT so.store_id) AS [Distinct Store Count],
    COUNT(so.order_id) AS [Total Order Count]
FROM sales.orders AS so
INNER JOIN sales.customers AS sc
    ON so.customer_id = sc.customer_id
GROUP BY sc.customer_id, sc.first_name, sc.last_name
HAVING COUNT(DISTINCT so.store_id) >= 2
ORDER BY
    [Distinct Store Count] DESC,
    [Total Order Count] DESC;
```

Question 4

List the number of orders each employee received in 2018. Employees who received no orders in 2018 should also appear with 0. The result should include the employee ID, first name, last name and order count. Sort by order count in descending order.

```
SELECT
    ss.staff_id AS [Employee ID],
    ss.first_name AS [First Name],
    ss.last_name AS [Last Name],
    COUNT(so.order_id) AS [Order Count]
FROM sales.staffs AS ss
LEFT JOIN sales.orders AS so
    ON ss.staff_id = so.staff_id
    AND so.order_date >= '20180101'
    AND so.order_date < '20190101'
GROUP BY ss.staff_id, ss.first_name, ss.last_name
ORDER BY [Order Count] DESC;
```

Question 5

List the employees who received at least one order in 2017 but no orders in 2018. The result should include the employee ID, first name, last name and number of orders in 2017. Sort by order count in descending order.

Solution 1

```
SELECT
    ss.staff_id AS [Employee ID],
    ss.first_name AS [First Name],
    ss.last_name AS [Last Name],
    COUNT(so.order_id) AS [2017 Order Count]
FROM sales.orders AS so
INNER JOIN sales.staffs AS ss
    ON so.staff_id = ss.staff_id
    AND so.order_date >= '20170101'
    AND so.order_date < '20180101'
LEFT JOIN sales.orders AS so2
    ON so.staff_id = so2.staff_id
    AND so2.order_date >= '20180101'
    AND so2.order_date < '20190101'
GROUP BY ss.staff_id, ss.first_name, ss.last_name
HAVING COUNT(so.order_id) >= 1 AND COUNT(so2.order_id) = 0
ORDER BY [2017 Order Count] DESC;
```

Solution 2

```
SELECT
    ss.staff_id AS [Employee ID],
    ss.first_name AS [First Name],
    ss.last_name AS [Last Name],
    COUNT(so.order_id) AS [2017 Order Count]
FROM sales.staffs AS ss
INNER JOIN sales.orders AS so
    ON ss.staff_id = so.staff_id
    AND so.order_date >= '20170101'
    AND so.order_date < '20180101'
LEFT JOIN sales.orders AS so2
    ON ss.staff_id = so2.staff_id
    AND so2.order_date >= '20180101'
    AND so2.order_date < '20190101'
WHERE so2.order_id IS NULL
GROUP BY ss.staff_id, ss.first_name, ss.last_name
ORDER BY [2017 Order Count] DESC;
```

Question 6

List the products whose list price is higher than the average list price of the products in their own category. The result should include the product name, category name and list price. Sort by price in descending order.

```
SELECT
    pp.product_name AS [Product Name],
    pc.category_name AS [Category Name],
    pp.list_price AS [List Price]
FROM production.products AS pp
INNER JOIN production.categories AS pc
    ON pp.category_id = pc.category_id
WHERE pp.list_price > (
    SELECT AVG(p2.list_price)
    FROM production.products AS p2
    WHERE p2.category_id = pp.category_id
)
ORDER BY pp.list_price DESC;
```

Question 7

List the products that are cheaper than the average product price of their own brand. The result should include the product name and list price; sort by price in ascending order. Use only the production.products table.

```
SELECT
    pp.product_name,
    pp.list_price
FROM production.products AS pp
WHERE pp.list_price < (
    SELECT AVG(p2.list_price)
    FROM production.products AS p2
    WHERE p2.brand_id = pp.brand_id
)
ORDER BY pp.list_price ASC;
```

Question 8

List the products that are more expensive than the average price of the products that belong to both their own brand and their own category. The result should include the product ID, product name and list price. Sort by price in descending order. Use only the production.products table.

```
SELECT
    pp.product_id,
    pp.product_name,
    pp.list_price
FROM production.products AS pp
WHERE pp.list_price > (
    SELECT AVG(p2.list_price)
    FROM production.products AS p2
    WHERE p2.brand_id = pp.brand_id
        AND p2.category_id = pp.category_id
)
ORDER BY pp.list_price DESC;
```

SQL Query Exercises 2

The queries in this document were written in Microsoft SQL Server (T-SQL) using the BikeStores sample database.

Mert Zeki Öztürk